«ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра комп'ютерної інженерії

# ЗВІТ
з лабораторної роботи № 2
з дисципліни «Програмування»

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Варіант № 3

Виконала:
Студентка 1-го курсу, групи КІ-26-1
Спеціальність: 123 «Комп'ютерна інженерія»
Неборачко Анастасія Іванівна

Перевірив:
Викладач кафедри комп'ютерної інженерії

Лисенко Максим Сергійович

Житомир – 2026

## 2.1 Мета роботи

Набути практичних навичок проектування комбінованих алгоритмів із використанням операторів розгалуження (if-else, тернарного оператора) та оператора множинного вибору (switch-case).

Дослідити алгоритмічні особливості та практичну реалізацію циклів із передумовою (while), післяумовою (do-while) та параметричних циклів (for).

Опанувати чисельні методи наближеного обчислення функцій за допомогою степеневих рядів та рекурентних співвідношень.

Закріпити навички системного контролю версій Git: робота з файлом .gitignore, семантичні коміти за стандартом Conventional Commits.

Опанувати академічні стандарти підготовки технічної документації шляхом формування обов'язкової пояснювальної записки у форматі PDF.

Індивідуальне завдання (Варіант № 3)
Розрахунок номера варіанта за порядковим номером у списку групи (N = 18):

$$Варіант = (18 - 1)\ mod\ 15 + 1 = 17\ mod\ 15 + 1 = \mathbf{3}$$

### Завдання 1. Табулювання кусочно-неперервної функції (task1.c)

Розробити програму мовою C для табулювання кусочно-неперервної функції Y(X) на заданому проміжку $[X_{start}, X_{end}]$ із кроком dX.
Математичний вираз кусочної функції для Варіанта 3:

$$Y(X) = \begin{cases} 2X + 1, & X \le 1 \\ \dfrac{X}{X - 2}, & 1 < X \le 3 \\ \sqrt{X}, & X > 3 \end{cases}$$

**Завдання 2. Наближене обчислення функцій чисельним рядом (task2.c)**

Розробити програму мовою С для обчислення значення функції cos(X) за допомогою розкладу в нескінченний степеневий ряд Маклорена із заданою інженерною точністю $\epsilon$.

Математичний вираз для Варіанта 3:

$$\cos(X) = \sum_{n=0}^{\infty} \frac{(-1)^n X^{2n}}{(2n)!} = 1 - \frac{X^2}{2!} + \frac{X^4}{4!} - \frac{X^6}{6!} + \dots$$

**3.1. Аналіз ОДЗ та точок розриву (Завдання 1)**

Кусочно-неперервна функція для Варіанта 3 задана виразом:

$$Y(X) = \begin{cases} 2X + 1 & , X \le 1 \\ \dfrac{X}{X-2}, & 1 < X \le 3 \\ \sqrt{X}, & X > 3 \end{cases}$$

Проведемо математичний аналіз області допустимих значень (ОДЗ) за інтервалами:

Інтервал 1 ($X \le 1$):

Вираз Y(X) = 2X + 1 є лінійною функцією.

Не містить операцій ділення чи добування кореня.

Визначений для всіх $X \in (-\infty; 1]$.

Інтервал 2 ($1 < X \le 3$):

Вираз $Y(X) = \frac{X}{X-2}$ є дробово-раціональною функцією.

Знаменник перетворюється на нуль при$X - 2 = 0 \Longrightarrow X = 2$.

Точка X = 2 належить даному півінтервалу (1; 3].

Висновок: Точка X = 2 є точкою розриву другого роду (ділення на нуль, функція прямує до нескінченності). У програмі передбачено перевірку$|X - 2| < \epsilon$ з виведенням повідомлення

Інтервал 3 ($X > 3$):

Вираз $Y(X) = \sqrt{X}$ вимагає виконанння умови підкореневого виразу $X \geq 0$.

Оскільки інтервал обмежений знизу умовою X > 3, усі значення X є строго додатними (X > 3 > 0).

Висновок: На цьому інтервалі функція визначена повністю.

### 3.2. Аналітичний вивід рекурентного співвідношення (Завдання 2)

Для обчислення cos(X) використовується ряд Маклорена:

$$\cos(X) = \sum_{n=0}^{\infty} \frac{(-1)^n X^{2n}}{(2n)!} = 1 - \frac{X^2}{2!} + \frac{X^4}{4!} - \frac{X^6}{6!} + \ldots$$

Запишемо загальний n-й член ряду та попередній (n-1)-й член:

$$a_n = \frac{(-1)^n \cdot X^{2n}}{(2n)!}$$

$$a_{n-1} = \frac{(-1)^{n-1} \cdot X^{2(n-1)}}{(2(n-1))!} = \frac{(-1)^{n-1} \cdot X^{2n-2}}{(2n-2)!}$$

Знайдемо коефіцієнт переходу $R(n, X) = \frac{a_n}{a_{n-1}}$$:

$$R(n, X) = \frac{\frac{(-1)^n \cdot X^{2n}}{(2n)!}}{\frac{(-1)^{n-1} \cdot X^{2n-2}}{(2n-2)!}} = \frac{(-1)^n}{(-1)^{n-1}} \cdot \frac{X^{2n}}{X^{2n-2}} \cdot \frac{(2n-2)!}{(2n)!}$$

Розпишемо та скорочимо кожен із трьох множників:

Знак: $\frac{(-1)^n}{(-1)^{n-1}} = (-1)^{n-(n-1)} = (-1)^1 = -1$

Степінь: $\frac{X^{2n}}{X^{2n-2}} = X^{2n-(2n-2)} = X^2$

Факторіал: $\frac{(2n-2)!}{(2n)!} = \frac{(2n-2)!}{(2n-2)!\cdot(2n-1)\cdot 2n} = \frac{1}{(2n-1)\cdot 2n}$

Підставляємо скорочення у формулу коефіцієнта переходу:

$$R(n, X) = -1 \cdot X^2 \cdot \frac{1}{(2n-1)\cdot 2n} = -\frac{X^2}{2n(2n-1)}$$

Остаточне рекурентне співвідношення: $a_0 = 1$

$$a_n = a_{n-1} \cdot \left(-\frac{X^2}{2n(2n-1)}\right), \quad n \geq 1$$

**4.1. Логіка роботи та захисні заходи у task1.c**

Програма призначена для табулювання кусочно-неперервної функції Y(X)на проміжку $[X_{start}, X_{end}]$ із кроком dX.

Валідація вхідних даних:

Програма зчитує змінні $X_{start}$, $X_{end}$ та dX за допомогою функцій безпечного введення.

Перевіряється некоректність меж та кроку: якщо$X_{start} \geq X_{end}$ або $dX \leq 0$, програма виводить аварійне повідомлення у потік stderr і завершує роботу з ненульовим кодом повернення (return 1).

Організація циклу табулювання:

Використовується цикл while із плаваючою змінною x, яка ініціалізується значенням $x_start$.

Для запобігання передчасному зупиненню циклу через похибки представлення чисел з плаваючою крапкою (IEEE 754) умова зупинки задається як x <= x_end + eps, де eps = dX / 1000.0.

Логіка розгалуження та захист від точок невизначеності:

На кожній ітерації використовується каскадна структура if - else if - else для визначення відповідного інтервалу:

-$X \leq 1.0$: $Обчислюється y = 2X + 1$.

-$1.0 < X \leq 3.0$: $Оскільки в точці X = 2$ виникає ділення на нуль, перед обчисленням виконується перевірка

fabs (x - 2.0) < 1e-7. Якщо умова виконується, прапорець is_defined встановлюється в 0. Інакше обчислюється$y = \frac{X}{X-2}$.

-$X > 3.0$: $Обчислюється\, y = \sqrt{X}$.

Якщо is_defined == 0, програма не виконує неприпустимих арифметичних операцій, а виводить у таблицю текстовий маркер .

## Форматоване виведення:

Для формування симетричної консольної таблиці використовуються модифікатори специфікаторів формату %10.4lf та %14.4lf, які фіксують ширину колонок та точність виведення дрібної частини.

### 4.2. Логіка роботи та захисні заходи у task2.c

Програма виконує чисельне додавання нескінченного ряду Маклорена для функції cos(X) до досягнення заданої інженерної точності $\epsilon$.

## Валідація та перевірка області збіжності:

Зчитуються значення X та точності $\epsilon$.

Перевіряється умова $\epsilon > 0$.

Оскільки область збіжності ряду длясos(X) є всією числовою віссю ( $-\infty < X < \infty$), введений аргумент X є завжди допустимим.

## Рекурентне обчислення та накопичення суми:

Перед початком циклу ініціалізується початковий доданок term = 1.0 ($a_0 = 1$), сума sum = 1.0 та лічильник ітерацій n = 0.

Цикл while (fabs(term) >= eps) виконує обчислення чергових членів ряду.

На кожному кроці n ($n = 1,2,3, \ldots$)доданок$a_n$обчислюється через рекурентний коефіцієнт R(n, X):

```
double r_n = - (x * x) / ((2.0 * n) * (2.0 * n - 1.0));
term = term * r_n;
sum += term;
```

Такий підхід повністю виключає використання важких функцій pow() та обчислення факторіала O(n), зводячи обчислення до 3 арифметичних операцій на ітерацію.

## Апаратний захист від зациклення (Timeout Guard):

Всередині циклу відстежується кількість ітерацій n.

Якщо n > MAX_ITERATIONS ($100\,000$), цикл примусово переривається інструкцією break, а користувачеві виводиться застереження про можливе розбігання ряду або надто мале значення $\epsilon$.

## Порівняльний аналіз та обчислення похибки:

Після завершення циклу обчислюється еталонне значення за допомогою бібліографічної функції cos(x) із <math.h>.

Обчислюється абсолютна похибка: abs_error = fabs(sum - exact_value).

Результати виводяться на екран із високою точністю для верифікації збіжності.

### 5.1. Повний лістинг файлу task1.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void) {
   double x_start, x_end, dx;

   printf("=== Task 1: Tabulation (Variant 3) ===\n");
   printf("Enter X_start, X_end, dX: ");

   if (scanf("%lf %lf %lf", &x_start, &x_end, &dx) != 3) {
      fprintf(stderr, "Error: invalid input format.\n");
      return 1;
   }

   if (x_start >= x_end || dx <= 0.0) {
      fprintf(stderr, "Error: X_start must be < X_end and dX > 0.\n");
      return 1;
   }

   printf("\n+------------+----------------+\n");
   printf("|     X      |      Y(X)      |\n");
   printf("+------------+----------------+\n");

   double x = x_start;
   double eps = dx / 1000.0;

   while (x <= x_end + eps) {
      double y = 0.0;
      int is_defined = 1;

      if (x <= 1.0) {
         y = 2.0 * x + 1.0;
      } else if (x <= 3.0) {
         if (fabs(x - 2.0) < 1e-7) {
            is_defined = 0;
         } else {
            y = x / (x - 2.0);
         }
      } else {
```

```
            if (x < 0.0) {
                is_defined = 0;
            } else {
                y = sqrt(x);
            }
        }

        if (is_defined) {
            printf("| %10.4lf | %14.4lf |\n", x, y);
        } else {
            printf("| %10.4lf |  [Undefined]  |\n", x);
        }

        x += dx;
    }

    printf("+------------+----------------+\n");
    return 0;
}
```

### 5.2. Повний лістинг файлу task2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX_ITERATIONS 100000

int main(void) {
    double x, eps;

    printf("=== Task 2: Approximating cos(X) (Variant 3) ===\n");
    printf("Enter X and eps: ");

    if (scanf("%lf %lf", &x, &eps) != 2) {
        fprintf(stderr, "Error: invalid input.\n");
        return 1;
```

```
    }

    if (eps <= 0.0) {
        fprintf(stderr, "Error: eps must be > 0.\n");
        return 1;
    }

    double sum = 0.0;
    double term = 1.0;
    int n = 0;

    sum += term;

    while (fabs(term) >= eps) {
        n++;

        if (n > MAX_ITERATIONS) {
            printf("\nWARNING: Max iterations reached (%d).\n",
MAX_ITERATIONS);
            break;
        }

        double r_n = - (x * x) / ((2.0 * n) * (2.0 * n - 1.0));
        term = term * r_n;
        sum += term;
    }

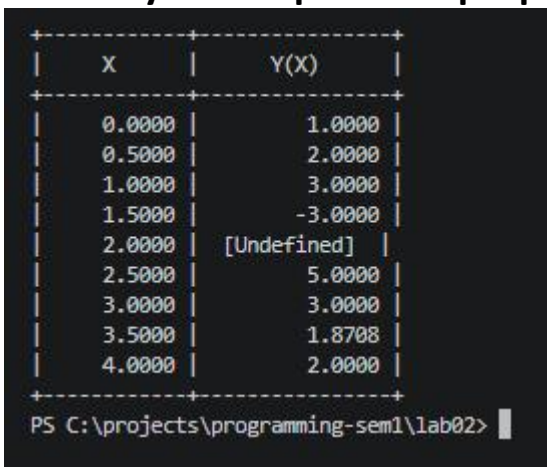
    double exact_value = cos(x);
    double abs_error = fabs(sum - exact_value);

    printf("\n--- Results ---\n");
    printf("X:           %.6lf\n", x);
    printf("eps:         %.8lf\n", eps);
    printf("Sum:         %.10lf\n", sum);
    printf("cos(x) math:  %.10lf\n", exact_value);
```

```
    printf("Abs Error:    %.10e\n", abs_error);
    printf("Iterations:   %d\n", n);

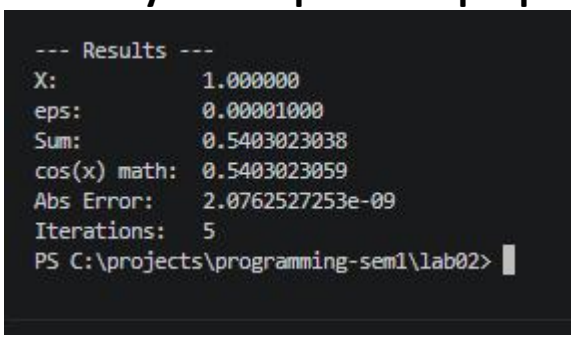
    return 0;
}
```

### 6.1. Результати роботи програми task1.c




Результати табулювання кусочно-неперервної функції.

### 6.2. Результати роботи програми task2.c




Результати обчислення cos(X) через ряд Маклорена.

## 7. ВИСНОВКИ

У ході виконання лабораторної роботи № 2 було здобуто практичні навички розробки алгоритмів із розгалуженими та циклічними процесами мовою С, а також опановано чисельні методи наближеного обчислення функцій за допомогою степеневих рядів.

Основні результати роботи:

1. Табулювання кусочно-неперервної функції (task1.c):
   - Реалізовано програму для табулювання функції Y(X) на заданому інтервалі [0; 4] із кроком dX = 0.5.
   - Забезпечено коректну обробку точки розриву другого роду (X = 2.0, ділення на нуль): програма уникнула аварійного завершення та вивела інформативне повідомлення [Undefined].
   - Застосовано захисні механізми валідації вхідних даних (перевірка X_start < X_end та dX > 0). 2.

2. Наближене обчислення cos(X) чисельним рядом (task2.c):
   - Опановано чисельний метод обчислення значення cos(X) за допомогою розкладу у ряд Маклорена.
   - Завдяки аналітичному виводу рекурентного співвідношення $R(n, X) = -X^2 / (2n \cdot (2n - 1))$ вдалося уникнути використання важких функцій pow() та обчислення факторіала на кожній ітерації, що суттєво підвищило обчислювальну ефективність програми.
   - Впроваджено апаратний захист від зациклення (Timeout Guard до 100 000 ітерацій) та перевірку інженерної точності eps > 0.
   - При випробуванні на значеннях X = 1.0 та eps = 0.00001 досягнуто абсолютної похибки порядку ~2.07e-09 всього за 5 ітерацій, що підтверджує високу швидкість збіжності даного ряду. 3.

3. Практичні навички:
   - Закріплено навички роботи з системою контролю версій Git та дотримання стандарту семантичних комітів (Conventional Commits).
   - Оформлено інженерну звітність відповідно до вимог ЄСДП та академічних стандартів.

Мету лабораторної роботи досягнуто в повному обсязі.